

Valdi: Value Diffusion World Models

Christopher Lindenberg, Kashyap Chitta

Keywords: Diffusion Models, World Models, Value Learning, Model Predictive Control.

Summary

World models can enable inference-time planning by imagining future trajectories, but their practical application requires dynamics prediction that is both fast enough for online use and expressive enough to represent uncertain futures. Diffusion models offer a natural mechanism for modeling such uncertain dynamics, yet their iterative inference procedure makes them difficult to combine with low-latency latent planning. We introduce Value Diffusion World Models (Valdi), an algorithm that jointly trains a latent diffusion dynamics model with objectives for representation learning, reward prediction, and value prediction in an online control loop. In preliminary experiments on the CarRacing environment, we find that using a single diffusion step at both training and inference time is sufficient for Valdi to match the control performance of a deterministic MLP dynamics baseline similar to TD-MPC. However, we show that increasing the inference-time diffusion steps results in more visually diverse, multi-modal trajectory predictions, but in turn degrades task performance in this simple environment. Analysis of the predicted value function suggests that, although diffusion rollouts incur larger short-horizon errors, their value estimates are better grounded than those of the MLP baseline at the longer-horizon rollout depth used for planning.

Contribution(s)

1. We propose Value Diffusion World Models (Valdi), a TD-MPC-style algorithm, trained end-to-end and online, that uses a latent diffusion model as its dynamics model in place of TD-MPC’s MLP.
Context: Diffusion models are a state-of-the-art approach to world modeling, but are typically applied in pixel space together with model-based RL (Alonso et al., 2024; Hafner et al., 2025), where efficiency requirements are not as high as MPC. Online latent planners in the TD-MPC family instead rely on a deterministic MLP dynamics model (Hansen et al., 2022; 2023). A latent diffusion dynamics model trained end-to-end inside such a value-based online planning loop has, to our knowledge, not been demonstrated.
2. We show that a single diffusion step at both training and inference time is sufficient for Valdi to match the MLP baseline in control performance.
Context: Diffusion models ordinarily require many iterative denoising steps, the primary reason their inference is considered incompatible with low-latency planning. Whether a single denoising step can retain control performance competitive with a deterministic one-step dynamics model has not, to our knowledge, been examined in the setting of latent MPC.
3. Increasing inference-time diffusion steps results in more visually diverse, multimodal predictions but degrades planning, exposing a tension between predictive expressivity and control quality in this setting.
Context: Diffusion world models are commonly motivated by their capacity to represent multimodal, uncertain futures (Agarwal et al., 2025), with the implicit assumption that richer predictive modeling benefits downstream control. To our knowledge, the effect of increasing this generative expressivity on planning performance within a value-based MPC loop has not been characterized.

Valdi: Value Diffusion World Models

Christopher Lindenberg¹, Kashyap Chitta²

christopher-sebastian.lindenberg@student.uni-tuebingen.de,
kashyap@kesai.eu

¹University of Tübingen, Germany

²KE:SAI – Kyutai ELLIS Scalable Autonomous Intelligence, Germany

Abstract

World models can enable Model Predictive Control (MPC), but this requires dynamics prediction that is both fast enough for online use and expressive enough to represent uncertain futures. Diffusion models offer a natural mechanism for modeling uncertain dynamics, yet their iterative inference procedure makes them difficult to use for low-latency latent planning. We bridge this gap with Value Diffusion World Models (**Valdi**), combining end-to-end online training for MPC with a latent diffusion dynamics model. In preliminary experiments on the CarRacing environment, we show that **Valdi**, using a single diffusion step at both training and inference, matches a deterministic MLP baseline. Our experiments expose a trade-off between predictive multimodality and control performance in this setup. Code is available at <https://github.com/Kit115/ValueDiffusionWorldModels>.

1 Introduction

Learned models of environment dynamics, also called *world models*, enable inference-time planning methods such as Model Predictive Control (MPC) (Garcia et al., 1989). To run online, MPC requires rapid inference, favoring lightweight *latent* world models that predict compressed representations of observations rather than raw pixels (Hansen et al., 2022). Latent world models, however, are prone to representation collapse due to trivial solutions in their optimization space (Maes et al., 2026).

In parallel, diffusion models (Ho et al., 2020) have emerged as a strong candidate for world modeling, accurately capturing complex distributions over long horizons (Agarwal et al., 2025). They are typically trained in observation (e.g., pixel) space to sidestep representation collapse (Alonso et al., 2024; Ho et al., 2022), yet their iterative inference is in tension with the low-latency requirements of MPC.

In this work, we bridge this gap with Value Diffusion World Models (**Valdi**, illustrated in Figure 1), an algorithm that trains a latent diffusion dynamics model to predict value functions, end-to-end and in an online control loop, in the style of TD-MPC (Hansen et al., 2022; 2023).

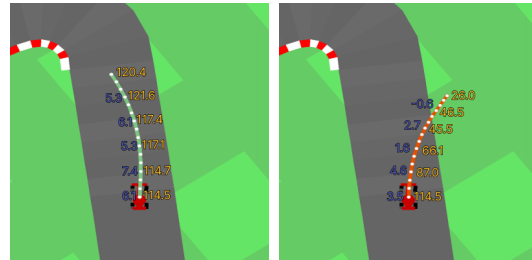


Figure 1: **Valdi** is a diffusion model that predicts action-sequence conditioned rewards and values, enabling Model Predictive Control. We show its predictions for a good and bad action sequence in the CarRacing environment (Towers et al., 2024).

2 Related Work

Diffusion World Models. Both pixel-space and latent world models have been used for training Reinforcement Learning (RL) agents inside of them, often referred to as model-based RL (Ha & Schmidhuber, 2018; Hafner et al., 2020; 2023). In this setting, diffusion models have emerged as a state-of-the-art approach for world modeling (Alonso et al., 2024; Hafner et al., 2025). Originally introduced for generating high dimensional natural data such as images or videos, diffusion models are able to represent highly complex, multi-modal data distributions, making them a natural choice for environments with uncertain state transitions (Agarwal et al., 2025; Gao et al., 2024; Yang et al., 2025). Although these methods demonstrate high visual fidelity, autoregressive image generation is hard to apply to planning, since diffusing high-dimensional data is prohibitively expensive. Further, the restriction to input space makes it unclear how to adopt multi-sensor setups, e.g., in autonomous vehicles, where a variety of sensors as well as proprioception are used (Chen et al., 2024). In contrast to prior pixel-space diffusion world models like DIAMOND (Alonso et al., 2024), Valdi diffuses compact latent states, enabling both compute-efficient control and multiple input signal types.

Temporal Difference Learning for Model Predictive Control (TD-MPC). This approach, which is most closely related to ours, pioneered the learning of a *Task-Oriented Latent Dynamics Model* (TOLD) for online planning in a latent space (Hansen et al., 2022; 2023). Given an observation s_t , its latent representation $\mathbf{z}_t = E_\theta(s_t)$, and the corresponding action taken $\mathbf{a}_t$, TOLD uses analogous high-level components to our model in Section 3: a representation model, a dynamics model, a reward model, and a value model (details in Supp. A). The main architectural difference is the dynamics model: TD-MPC predicts a single next latent state $\mathbf{z}_{t+1}$ from $(\mathbf{z}_t, \mathbf{a}_t)$ and rolls this one-step transition out iteratively to construct imagined trajectories. At inference time, like TD-MPC, we plan over latent rollouts scored by discounted predicted rewards plus a terminal value, executing only the first action before replanning. We optimize this with the Cross Entropy Method (CEM) rather than TD-MPC’s MPPI (Williams et al., 2017)(Supp. D). A limitation of TD-MPC is that its dynamics model is a deterministic MLP that lacks an explicit mechanism for representing ambiguous or multi-modal futures. This is notable given that PlaNet (Hafner et al., 2019), an early influence on latent planning of this kind, already included a stochastic component. Valdi builds on the same TD-MPC-style value-guided MPC structure, but replaces the deterministic latent transition model with a diffusion dynamics model, providing this previously missing capability.

3 Value Diffusion World Models

Diffusion-Based TOLD. Like TD-MPC’s TOLD, Valdi consists of four components (Figure 2):

$$\begin{aligned} \textbf{Representation: } \mathbf{z}_t &= E_\theta(s_t) \\ \textbf{Dynamics: } \hat{\mathbf{u}}_{t+1:t+H}^\tau &= D_\theta(\mathbf{z}_t, \mathbf{z}_{t+1:t+H}^\tau, \mathbf{a}_{t:t+H-1}, \tau) \\ \textbf{Reward: } \hat{r}_t &= R_\theta(\mathbf{z}_t, \mathbf{a}_t) \\ \textbf{Value: } \hat{v}_t &= V_\theta(\mathbf{z}_t) \end{aligned}$$

First, rather than rolling out latent states autoregressively, Valdi exploits the diffusion model to generate the entire H -step latent trajectory jointly. We use the velocity parameterization of Salimans & Ho (2022), where $\hat{\mathbf{u}}_{t+1:t+H}^\tau$ is the velocity for the noised latent trajectory $\mathbf{z}_{t+1:t+H}^\tau$ at diffusion timestep τ . Second, we predict state values (with a value head V_θ) rather than action values: Valdi, unlike TD-MPC, has no policy prior, so this simpler formulation suffices (full justification in Supp. A). Architecturally, E_θ is a CNN, R_θ and V_θ are MLPs, and D_θ is a bidirectional encoder-only transformer, totaling $\sim 5\text{M}$ parameters (Supp. B).

Training. We sample trajectories $(s_{t:t+H}, \mathbf{a}_{t:t+H-1}, r_{t:t+H-1})$ from a replay buffer and optimize $\mathcal{L}_{\text{diff}} + \mathcal{L}_{\text{rew}} + \mathcal{L}_{\text{val}}$, with additional regularization, see Supp. C. The first component, $\mathcal{L}_{\text{diff}}$, is the

standard latent diffusion objective. Crucially, and unlike nearly all latent diffusion models, which freeze their encoders, we **jointly train** E_θ with the other components. For the reward and value losses, we obtain one-step denoised predictions from noised state embeddings $\bar{\mathbf{z}}_{t+1:t+H}^\tau$ via:

$$\hat{\mathbf{z}}_{t+1:t+H} = \sqrt{\alpha^\tau} \bar{\mathbf{z}}_{t+1:t+H}^\tau - \sqrt{1 - \alpha^\tau} \hat{\mathbf{u}}_{t+1:t+H}^\tau, \quad (1)$$

where $\mathbf{z}$, $\mathbf{z}^\tau$, and $\hat{\mathbf{z}}$ denote clean, noisy, and denoised latents, and a bar marks quantities encoded by an Exponential Moving Average (EMA) target encoder $E_{\bar{\theta}}$. We then apply a TD-MPC-style reward error $\mathcal{L}_{\text{rew}}$ and a temporal difference loss $\mathcal{L}_{\text{val}}$ (Sutton, 1988) to the denoised predictions $\hat{\mathbf{z}}_{t+1:t+H}$ (details and pseudo-code in Supp. C). The one-step denoising estimate keeps training tractable and matches our single-step inference for low-latency MPC; we revisit this choice in Section 4.

Inference. Our Cross Entropy Method solver (Rubinstein & Kroese, 2004) maximizes discounted returns $\gamma^H V_\theta(\hat{\mathbf{z}}_H) + \sum_{h=0}^{H-1} \gamma^h R_\theta(\hat{\mathbf{z}}_h, \mathbf{a}_h)$ over $\mathbf{a}_{0:H-1}$ and executes the first action $\mathbf{a}_0^*$ (Supp. D).

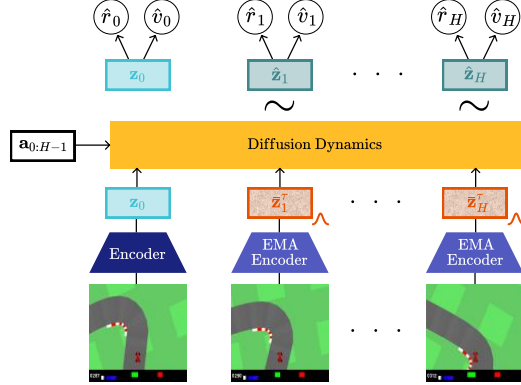


Figure 2: **Method.** Vald encodes states, conditions diffusion dynamics on an action sequence and noised latents from a target encoder, and uses the denoised latent trajectory for reward and value prediction.

4 Experiments

As a preliminary proof of concept, all of our experiments use a modified version of the well studied CarRacing environment (see Supp. E for details). We are interested in the following questions: (1) How does the **performance** of Vald compare to a standard MLP baseline? (2) Does the diffusion dynamics model retain its capacity for **multimodal predictions** when trained jointly with the rest of the system? (3) How does the choice of dynamics model affect the **value function**? We train all models on a single RTX 4080 GPU, with hyperparameters listed in Supp. F.

Performance and Multimodality. We compare Vald against a baseline that swaps the diffusion dynamics for a deterministic one-step MLP, similar to TD-MPC, with all other parameters identical. Each system is trained twice and evaluated on 100 fixed tracks. Unlike the MLP, Vald has one additional axis for scaling inference-time compute: the number of diffusion steps. Training and our default evaluation use a single step. In Figure 3 we explore multi-step inference using a deterministic DDIM sampler (Song et al., 2020).

Further, we extend this experiment to probe predictive variety. We exploit CarRacing’s partial observability: the upcoming track is hidden, so given a frame and action sequence, no unique future observation stream is predictable. A deterministic model must therefore output a mean or commit to a single mode, while a diffusion model can in principle represent a distribution over plausible continuations. We train a pixel decoder post-hoc on frozen encoder latents, generate $N = 100$ rollouts

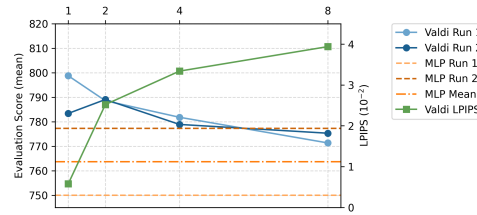


Figure 3: **Performance and multimodality.** Across two runs, more inference diffusion steps do not improve control over our single-step default, but substantially increase the visual variety (LPIPS) among generated futures.

for several $(s_0, \mathbf{a}_{0:H-1})$ pairs, and report mean pairwise LPIPS (Zhang et al., 2018) between their decoded final states (visual examples in Supp. G.2). A higher LPIPS indicates more variety.

At one diffusion step, `Valdi` matches the MLP baseline in control within run-to-run variance (additional analysis in Supp. G). However, its predictive distribution is narrow, typically committing to a single track continuation. Counterintuitively, more diffusion steps slightly degrade control (also within run-to-run variance) while substantially increasing trajectory variety. We attribute the degradation to a training-inference mismatch combined with increased trajectory variance that the CEM solver is ill-equipped to exploit. The result exposes a tension: more diffusion steps yield richer, visually plausible variety, but this variety degrades control performance in this environment.

Value Function. Next, we examine how the choice of dynamics model affects the value and reward functions. We evaluate two properties of the value function along imagined rollouts: (1) *Self-consistency*: does the value function agree with itself across adjacent steps of an imagined rollout, in the sense of a small TD residual? (2) *Grounding*: does the value of an imagined latent agree with the value of the real latent the agent encounters at the same environment timestep? Equations for these terms are detailed in the following. As in Section 3, we use a hat ($\hat{z}$) for imagined latents and an unadorned symbol (z) for real, encoder-produced latents. h indexes depth into an imagined rollout and t denotes the environment timestep, with the convention $\hat{z}_0 = z_t$ at the start of each rollout.

Self-Consistency. We sample 10,000 real states z_t from a post-hoc collected on-policy dataset of each system (i.e., `Valdi` and the MLP baseline). Starting from $\hat{z}_0 = z_t$, we let the planner propose an action sequence $\mathbf{a}_{0:H-1}$ which the dynamics model uses to imagine a trajectory $\hat{z}_{0:H}$. We then compute the one-step TD residual δ_h^{TD} at each depth. A positive sign for self-consistency indicates that $V(\hat{z}_h)$ underestimates the bootstrap target of $R(\hat{z}_h, \mathbf{a}_h) + \gamma V(\hat{z}_{h+1})$:

$$\delta_h^{\text{TD}} = R(\hat{z}_h, \mathbf{a}_h) + \gamma V(\hat{z}_{h+1}) - V(\hat{z}_h). \quad (2)$$

Grounding. Self-consistency only tells us whether the value function agrees with itself; not whether the imagined latents resemble the real states the agent will actually encounter. To measure this, we roll out our policy in the environment and, at each step, also generate an imagined trajectory from the planner. This gives us, for each environment time t and rollout depth h , both an imagined latent $\hat{z}_h$ (starting from $\hat{z}_0 = z_t$) and a real latent z_{t+h} that the agent reaches h steps later in the real environment. We compute:

$$\delta_h^{\text{drift}} = V(z_{t+h}) - V(\hat{z}_h). \quad (3)$$

A negative value indicates an optimistic imagined latent: the world model expects more reward than reality delivers. We aggregate δ_h^{drift} over 100 closed-loop trajectories of 200 world model steps each.

We report results in Table 1. At short rollout depths, `Valdi` is less accurate than the MLP in terms of both self-consistency and grounding. As depth increases, however, `Valdi`’s error grows more slowly, and near the planner’s bootstrap depth H (self-consistency at $h = 4$, grounding at $h = 5$) our model is both more self-consistent and better grounded than the baseline. These gaps are small and within run-to-run variance, but the trend is consistent across both diagnostics, suggesting that the predictions that matter most for planning are the ones at which `Valdi` outperforms the MLP.

Table 1: **Value function diagnostics.** Signed errors δ at each rollout step h . **Bold** marks the model with a lower error, i.e., $|\delta|$ closer to zero.

		$h=0$	$h=1$	$h=2$	$h=3$	$h=4$
δ_h^{TD}	<code>Valdi</code>	3.28	2.17	2.00	1.93	1.95
	MLP	2.05	2.00	2.07	2.19	2.23
		$h=1$	$h=2$	$h=3$	$h=4$	$h=5$
δ_h^{drift}	<code>Valdi</code>	-1.53	-2.11	-2.56	-3.04	-3.52
	MLP	-0.62	-1.21	-1.93	-2.76	-3.65

5 Conclusion

We present Value Diffusion World Models, an algorithm that combines diffusion and latent world models for inference time planning. We show that a latent diffusion dynamics model can be trained

jointly with a value function inside a TD-MPC-style online loop, matching MLP control performance while producing multi-modal trajectory predictions.

We see two immediate limitations and corresponding directions for future work. First, the one-step estimate used at both training and inference is unlikely to scale to environments with substantially more complex dynamics, where multi-step denoising would likely be necessary to recover accurate predictions. To remain computationally feasible at inference, this may then require research into distillation from multi-step teachers to single-step students. Second, our results suggest that simply changing the number of inference-time diffusion steps degrades planning performance. Obtaining a model that is flexible at inference time via different diffusion schedules, e.g., enabling test-time scaling, will likely need a corresponding change in the training procedure.

References

- Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, Jiaojiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Samuel Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, Gergely Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixé, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezani, Fitsum Reda, Xiaowei Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne Tchapmi, Przemek Tredak, Wei-Cheng Tseng, Jibin Varghese, Hao Wang, Haoxiang Wang, Heng Wang, Ting-Chun Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Lin Yen-Chen, Xiaohui Zeng, Yu Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zólkowski. Cosmos world foundation model platform for physical AI. *arXiv.org*, 2501.03575, 2025.
- Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *NeurIPS*, 2024.
- Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. *arXiv.org*, 2025.
- Li Chen, Penghao Wu, Kashyap Chitta, Bernhard Jaeger, Andreas Geiger, and Hongyang Li. End-to-end autonomous driving: Challenges and frontiers. *PAMI*, 2024.
- Shenyuan Gao, Jiazhi Yang, Li Chen, Kashyap Chitta, Yihang Qiu, Andreas Geiger, Jun Zhang, and Hongyang Li. Vista: A generalizable driving world model with high fidelity and versatile controllability. In *NeurIPS*, 2024.
- Carlos E Garcia, David M Prett, and Manfred Morari. Model predictive control: Theory and practice—a survey. *Automatica*, 1989.
- David Ha and Jürgen Schmidhuber. World models. *arXiv.org*, 1803.10122, 2018.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018a.
- Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv.org*, 1812.05905, 2018b.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *ICML*, pp. 2555–2565. PMLR, 2019.

-
- Danijar Hafner, Timothy P. Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv.org*, 2023.
- Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv.org*, 2509.24527, 2025.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. *arXiv.org*, 2203.04955, 2022.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. *arXiv.org*, 2023.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. In *NIPS*, 2022.
- Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. In *ICLR*, 2016.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv.org*, 2603.19312, 2026.
- Reuven Y Rubinstein and Dirk P Kroese. The cross-entropy method: a unified approach to combinatorial optimization, monte-carlo simulation, and machine learning, 2004.
- Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *arXiv.org*, 2022.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv.org*, 2010.02502, 2020.
- Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv.org*, 2024.
- Grady Williams, Andrew Aldrich, and Evangelos A Theodorou. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 2017.
- Jiazhi Yang, Kashyap Chitta, Shenyuan Gao, Long Chen, Yuqian Shao, Xiaosong Jia, Hongyang Li, Andreas Geiger, Xiangyu Yue, and Li Chen. Resim: Reliable world simulation for autonomous driving. In *NeurIPS*, 2025.
- Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, pp. 586–595, 2018.

Supplementary Materials

The following content was not necessarily subject to peer review.

A Preliminaries

A.1 TD-MPC

TD-MPC trains a *Task-Oriented Latent Dynamics Model* (TOLD), which is then used for inference-time planning in latent space (Hansen et al., 2022; 2023). The TOLD model consists of a representation model E_θ that maps observations to latent states, a dynamics model D_θ that predicts the next latent state conditioned on the current latent state and action, a reward function R_θ , an action-value function Q_θ , and a prior policy π_θ .

The TOLD components are trained jointly with reward prediction, temporal-difference learning for the Q -function, and a latent consistency objective that encourages self-consistency over multi-step latent rollouts. The dynamics model in TD-MPC is parameterized as a deterministic MLP that directly predicts the next latent state:

$$\mathbf{z}_{t+1} = D_\theta(\mathbf{z}_t, \mathbf{a}_t). \quad (4)$$

This deterministic transition model is efficient, but it does not explicitly represent ambiguous futures introduced by partial observability or stochastic, multi-modal environment transitions.

At inference time, TD-MPC uses the learned model inside a standard MPC loop. It searches for an action sequence by solving

$$\mathbf{a}_{0:H-1}^* = \underset{\mathbf{a}_{0:H-1}, \mathbf{z}_{0:H}}{\operatorname{argmax}} \mathbb{E}[\mathcal{J}], \quad (5)$$

with trajectory score

$$\mathcal{J} = \gamma^H Q_\theta(\mathbf{z}_H, \pi_\theta(\mathbf{z}_H)) + \sum_{t=0}^{H-1} \gamma^t R_\theta(\mathbf{z}_t, \mathbf{a}_t), \quad (6)$$

subject to the latent dynamics constraint $\mathbf{z}_{t+1} = D_\theta(\mathbf{z}_t, \mathbf{a}_t)$ for each rollout step. As is common in MPC, the planner executes only the first action $\mathbf{a}_0^*$ and replans from the next observed environment state. TD-MPC uses Model-Predictive Path Integral (MPPI) control (Williams et al., 2017) as its MPC solver.

Key Distinctions. Our method differs from TD-MPC in some key aspects that are likely to result in non-negligible performance implications, the largest of which is our omission of the prior policy π_θ . TD-MPC and its successors use this prior policy to bootstrap the planning process with a small percentage of action sequences that are generated by interleaving actions proposed by π_θ with predictions from the dynamics model,

$$\mathbf{a}_0 \sim \pi_\theta(\mathbf{z}_0), \mathbf{z}_1 = D_\theta(\mathbf{z}_0, \mathbf{a}_0), \mathbf{a}_1 \sim \pi_\theta(\mathbf{z}_1), \mathbf{z}_2 = D_\theta(\mathbf{z}_1, \mathbf{a}_1), \mathbf{a}_2 \sim \pi_\theta(\mathbf{z}_2), \dots$$

which results in better planning efficiency and possibly asymptotic performance. Because our dynamics model jointly predicts all future states conditioned on the full action sequence, this interleaving is not naturally possible, so we do not bootstrap planning in this way.

The secondary purpose of π_θ is to provide an action for the bootstrap value in Equation 6 and to make Q_θ trainable via DDPG (Lillicrap et al., 2016) or SAC (Haarnoja et al., 2018a;b). We sidestep both uses by employing a state-value function V_θ in place of an action-value function Q_θ , which significantly reduces code complexity and removes moving parts from training. We do not formally ablate this choice in the present work, as the focus is the dynamics model and its effect on the resulting state values; in early exploration we found no meaningful performance difference between Q_θ and V_θ implementations. Crucially, we also use V_θ for the MLP baseline in Section 4 rather than

Table 2: Architecture of the shared and dynamics-specific components. The latent dimension is 64; the per-step action conditioning has dimension 9 (3 raw action dimensions $\times$ an action chunk of 3, see Supp. D). All nonlinearities are tanh-approximated GELU.

Component	Type	Details
E_θ (visual)	CNN	5 conv layers, channels $3 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, projected to a 256-d embedding
E_θ (proprio)	MLP	$7 \rightarrow 256 \rightarrow 256$
E_θ (head)	MLP	concat(512) $\rightarrow$ LayerNorm $\rightarrow 512 \rightarrow 64$ latent
D_θ (Valdi)	Transformer	6 pre-norm blocks, width 256, FFN $256 \rightarrow 512 \rightarrow 256$; final Layer-Norm + linear readout to 64 (velocity)
D_θ (baseline)	Residual MLP	input 73 $\rightarrow 384$, 6 residual blocks (FFN $384 \rightarrow 768 \rightarrow 384$), linear readout to 64
R_θ	Residual MLP	input 73 $\rightarrow 256$, 2 residual blocks (FFN $256 \rightarrow 512 \rightarrow 256$), linear readout to 1
V_θ	MLP ensemble	$2 \times [64 \rightarrow 512 \rightarrow 512 \rightarrow 1]$ (minimum taken over the two heads, see Supp. C)

switching to a Q_θ -plus- π_θ setup, to keep the comparison as clean as possible. Our method differs from TD-MPC in other, smaller respects (e.g., exploration-schedule parameters) that are unlikely to have a significant effect.

A.2 Diffusion Models

We train the dynamics model with the velocity-prediction diffusion objective

$$\mathcal{L}_{\text{diff}} = \left\| \mathbf{u}_{t+1:t+H}^\tau - \hat{\mathbf{u}}_{t+1:t+H}^\tau \right\|_2, \quad \text{where} \quad (7)$$

$$\begin{aligned} \mathbf{u}_{t+1:t+H}^\tau &= \sqrt{\alpha^\tau} \boldsymbol{\epsilon} - \sqrt{1 - \alpha^\tau} \bar{\mathbf{z}}_{t+1:t+H} \\ \hat{\mathbf{u}}_{t+1:t+H}^\tau &= D_\theta(\mathbf{z}_t, \bar{\mathbf{z}}_{t+1:t+H}^\tau, \mathbf{a}_{t:t+H-1}, \tau) \\ \bar{\mathbf{z}}_{t+1:t+H}^\tau &= \sqrt{\alpha^\tau} \bar{\mathbf{z}}_{t+1:t+H} + \sqrt{1 - \alpha^\tau} \boldsymbol{\epsilon}. \end{aligned}$$

Here $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$, and $\alpha^\tau = \prod_{i=1}^\tau (1 - \beta_i)$ is the cumulative product of the standard linear variance schedule (Ho et al., 2020), with β increasing linearly from $\beta_1 = 1\text{e-}4$ to $\beta_T = 2\text{e-}2$ over $T = 1000$ steps:

$$\beta_i = \beta_1 + \frac{i-1}{T-1} (\beta_T - \beta_1), \quad \alpha^\tau = \prod_{i=1}^\tau (1 - \beta_i). \quad (8)$$

B Architecture Details

Both Valdi and the MLP baseline share the TOLD layout described in Section 3: a representation model E_θ , a dynamics model D_θ , a reward model R_θ , and a value model V_θ . The two systems differ *only* in the dynamics model; every other component is identical, isolating the dynamics model as the single variable under study. We summarize all components in Table 2.

Valdi has 5,390,230 trainable parameters and the MLP baseline has 5,795,363. We note that the baseline is the slightly *larger* of the two, so any performance parity is not the result of giving Valdi a capacity advantage.

Encoder. The representation model encodes the two input modalities separately: the (proprioception-masked) frame through the convolutional stack and the seven-dimensional proprioceptive vector through a small MLP (see Supp. E for the modality split). The two 256-dimensional

embeddings are concatenated and passed through a LayerNorm-prefixed projection head to a 64-dimensional latent.

Dynamics. The `Valdi` dynamics model is a bidirectional, encoder-only transformer of 6 pre-norm blocks operating at width 256, with one token per world-model step. The diffusion timestep τ is mapped to a 32-dimensional vector by a learned embedding (`Embedding(1000, 32)`). Action and diffusion-timestep conditioning are injected by concatenating the per-step action chunk and the timestep embedding to the (noised) latent token along the channel dimension before the first block; a final LayerNorm and linear layer read out the 64-dimensional velocity per step. The baseline replaces this with a residual MLP of matched depth that maps the current latent and action chunk directly to the next latent.

Reward and Value. The reward model is a residual MLP that takes the latent concatenated with the action chunk ($64 + 9 = 73$). The value model is an ensemble of two identical MLP heads; following common practice we take the elementwise minimum of the two heads when forming TD targets to mitigate value overestimation (Supp. C).

C Training Details

Here we provide a detailed description of our algorithm in the form of pseudocode that closely aligns with our implementation (Algorithm 1).

Training Objective. We optimize the weighted sum of four terms,

$$\mathcal{L} = (1 - \lambda_{\text{reg}})(\lambda_{\text{diff}} \mathcal{L}_{\text{diff}} + \lambda_{\text{rew}} \mathcal{L}_{\text{rew}} + \lambda_{\text{val}} \mathcal{L}_{\text{val}}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}, \quad (9)$$

where $\mathcal{L}_{\text{diff}}$ is the velocity diffusion loss (Supp. A), $\mathcal{L}_{\text{rew}}$ is a TD-MPC-style reward error, and $\mathcal{L}_{\text{val}}$ is a Temporal Difference loss (Sutton, 1988); both $\mathcal{L}_{\text{rew}}$ and $\mathcal{L}_{\text{val}}$ are applied to the one-step denoised latents $\hat{\mathbf{z}}_{t+1:t+H}$. The three task losses are described in Section 3; loss weights are listed in Supp. F. We use the minimum over two value heads when forming the TD target, a standard trick to avoid value overestimation.

Latent Collapse and SIGReg. The fourth loss term, $\mathcal{L}_{\text{reg}}$, is a SIGReg regularizer (Balestriero & LeCun, 2025) that encourages an isotropic Gaussian on the latent distribution; we apply it only to the first timestep (t_0) of each sampled training trajectory, and refer the reader to (Balestriero & LeCun, 2025) for its full definition. We include SIGReg chiefly so that our reported configuration faithfully reflects our implementation, rather than as a principled component of the method. In our experiments we observed that both `Valdi` and the MLP baseline learn without collapsing whether or not SIGReg is enabled; we did not, however, complete full training runs at $\lambda_{\text{reg}} = 0$ due to resource constraints. We therefore make no claim that SIGReg is either necessary or the operative mechanism preventing collapse, and leave a careful study of which components ground the latent space to future work.

Algorithm 1 Diffusion-Based TOLD Training

Require: $\theta, \bar{\theta}$: online and target network parameters

Require: $\eta, \beta, \lambda, \mathcal{B}$: learning rate, EMA coefficient, loss weights, replay buffer

```
1: while not converged do  
  // Collect episode with MPC policy  
2:   for  $t = 0 \dots T$  do  
3:      $\mathbf{a}_t \sim \Pi_{\theta}^{\text{MPC}}(\cdot \mid E_{\theta}(\mathbf{s}_t))$  ▷ Sample action using MPC  
4:      $(\mathbf{s}_{t+1}, r_t) \sim \text{Env}(\mathbf{s}_t, \mathbf{a}_t)$  ▷ Step environment  
5:      $\mathcal{B} \leftarrow \mathcal{B} \cup \{\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}\}$  ▷ Store transition in Replay Buffer  
6:   end for  
  // Update model using data in replay buffer  
7:   for num updates per episode do  
8:      $\mathbf{s}_{t:t+H}, \mathbf{a}_{t:t+H-1}, r_{t:t+H-1} \sim \mathcal{B}$  ▷ Sample trajectory segment from Replay Buffer  
9:      $\mathbf{z}_{t:t+H}, \bar{\mathbf{z}}_{t:t+H} = E_{\theta}(\mathbf{s}_{t:t+H}), E_{\bar{\theta}}(\mathbf{s}_{t:t+H})$  ▷ Encode observations  
10:     $\tau, \epsilon \sim U(0, 1000), \mathcal{N}(0, I)$  ▷ Sample diffusion step and noise  
11:     $\bar{\mathbf{z}}_{t+1:t+H}^{\tau} \leftarrow \sqrt{\alpha^{\tau}} \bar{\mathbf{z}}_{t+1:t+H} + \sqrt{1 - \alpha^{\tau}} \epsilon$  ▷ Forward noising  
12:     $\mathbf{u}_{t+1:t+H}^{\tau} \leftarrow \sqrt{\alpha^{\tau}} \epsilon - \sqrt{1 - \alpha^{\tau}} \bar{\mathbf{z}}_{t+1:t+H}^{\tau}$  ▷ Velocity target  
13:     $\hat{\mathbf{u}}_{t+1:t+H}^{\tau} \leftarrow D_{\theta}(\mathbf{z}_t, \bar{\mathbf{z}}_{t+1:t+H}^{\tau}, \mathbf{a}_{t:t+H-1}, \tau)$  ▷ Predict denoising direction  
14:     $\hat{\mathbf{z}}_{t+1:t+H} \leftarrow \sqrt{\alpha^{\tau}} \bar{\mathbf{z}}_{t+1:t+H}^{\tau} - \sqrt{1 - \alpha^{\tau}} \hat{\mathbf{u}}_{t+1:t+H}^{\tau}$  ▷ Reconstruct latent rollout  
15:     $\mathcal{L} \leftarrow (1 - \lambda_{\text{reg}})(\lambda_{\text{diff}} \mathcal{L}_{\text{diff}} + \lambda_{\text{rew}} \mathcal{L}_{\text{rew}} + \lambda_{\text{val}} \mathcal{L}_{\text{val}}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}$  ▷ Total training loss  
16:     $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$  ▷ Update online network  
17:     $\bar{\theta} \leftarrow (1 - \beta) \bar{\theta} + \beta \theta$  ▷ Update target network  
18:   end for  
19: end while
```

D Planning and Inference

Due to its simplicity and more widespread use, at inference time, we plan in latent space with a Cross Entropy Method (CEM) solver (Rubinstein & Kroese, 2004) rather than the MPPI solver used by TD-MPC. Starting from the encoded current state $\hat{\mathbf{z}}_0 = E_{\theta}(\mathbf{s}_t)$, we sample a population of candidate action sequences from a diagonal Gaussian, generate the corresponding H -step latent trajectories with the dynamics model, score each by the discounted return

$$\gamma^H V_{\theta}(\hat{\mathbf{z}}_H) + \sum_{h=0}^{H-1} \gamma^h R_{\theta}(\hat{\mathbf{z}}_h, \mathbf{a}_h), \quad (10)$$

refit the sampling distribution to the elite set, and iterate. After the final CEM iteration we execute the first action and replan at the next environment step. The CEM population size, elite count, iteration budget, and initial (μ^0, σ^0) are listed in Supp. F.

Action Chunking. The world model operates at a coarser temporal resolution than the environment: a single world-model step corresponds to 3 environment steps (action chunk of 3). Each world-model transition is therefore conditioned on a chunk of 3 environment actions ($3 \times 3 = 9$ scalar action dimensions, hence the action conditioning of dimension 9 in Supp. B). With a world-model horizon of $H = 5$, the planner reasons over 5 world-model steps, equivalent to a 15-step environment horizon, while only paying for 5 dynamics evaluations.

Inference Cost. For Valdi, trajectory prediction dominates the planning cycle, so inference cost grows linearly in the number of diffusion steps. At our default of a single diffusion step, the planner runs at over 10 Hz on a single RTX 4080, and the agent acts at over 30 Hz in the environment by virtue of action chunking. The MLP baseline is slightly faster than Valdi; we note, however, that neither approach underwent significant performance optimization.

E Environment

Here we describe the modifications to the CarRacing environment that we employed in our experiments. We modify the environment so that visual and proprioceptive information are split into two separate inputs. In the original environment the proprioceptive information is rendered as a bar at the bottom of the frame; we black out that section of the visual input and instead expose the proprioceptive information directly as a seven-dimensional state vector. We do this primarily to explore multi-modal inputs for our algorithm, in anticipation of future work. Additionally, we truncate trajectories at 600 environment frames rather than the usual 1000.

F Hyperparameters

We list all hyperparameters in Table 3. The action-chunking entry is explained in Supp. D.

Hyperparameter	Value
Discount factor (γ)	0.99
Seed Trajectories	10
Replay buffer size	1000 Trajectories
Replay sampling technique	Uniform
Planning horizon (WM) (H)	5
Planning horizon (Env)	15
Action chunking	3 (i.e., model predicts $s_3, s_6, \dots, s_{15}$ conditioned on $a_{0:14}$)
Initial parameters (μ^0, σ^0)	(0, 1)
Population size	512
Elite samples	64
Iterations	10
Inference Diffusion Steps	1
Latent dimension	64
Learning rate (η)	3e-4
Optimizer (θ)	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Reward loss weight (λ_{rew})	0.01
Value loss weight (λ_{val})	0.01
Diffusion loss weight (λ_{diff})	1
Regularization loss weight (λ_{reg})	0.05
Exploration schedule (ϵ)	0.25 $\rightarrow$ 0.05 (250 trajectories max $\rightarrow$ 250 trajectories linear decay)
Batch size	256
Polyak/EMA coefficient (β)	0.005
Number of updates per trajectory	60 (update/data ratio $\frac{1}{10}$)
$\bar{\theta}$ update frequency	1

Table 3: Hyperparameters.

G Additional Results

G.1 Training and Evaluation Returns

We report the training-time and evaluation-time returns of both systems in Figure 4. Valdi matches the MLP baseline within run-to-run variance, neither significantly improving nor degrading control performance.

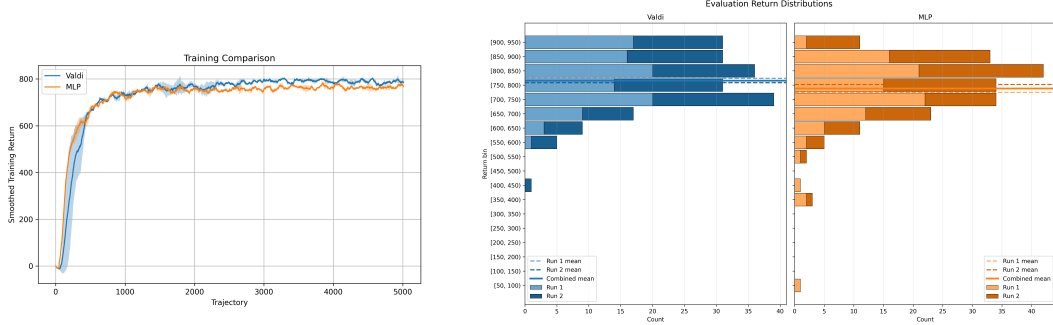


Figure 4: The returns that our system and our baseline obtain during training time (left). The performance both runs of both systems obtain at evaluation time (right).

G.2 Multimodal Trajectory Predictions

Here we provide an example of our dynamics model’s capacity to generate diverse futures from the same start state. We obtain the four trajectories shown by generating 100 imagined futures from a single starting state, computing pairwise visual similarity between their decoded final states, and selecting the subset of 4 trajectories that minimizes mutual similarity. We then decode each state of each trajectory independently.

Decoder. The visualizations and the LPIPS measurements in Section 4 rely on a pixel decoder trained *post hoc* on *frozen* encoder latents; the decoder is never used during training or planning. The decoder is a Vision Transformer that reconstructs image patches from a latent. We train one decoder per fully trained encoder, and each decoder is trained on an on-policy dataset collected by rolling out exactly the model whose latents it reconstructs. This ensures each decoder is trained on the latent distribution induced by its own model, so the decoded futures faithfully reflect that model’s predictions rather than an out-of-distribution mapping.

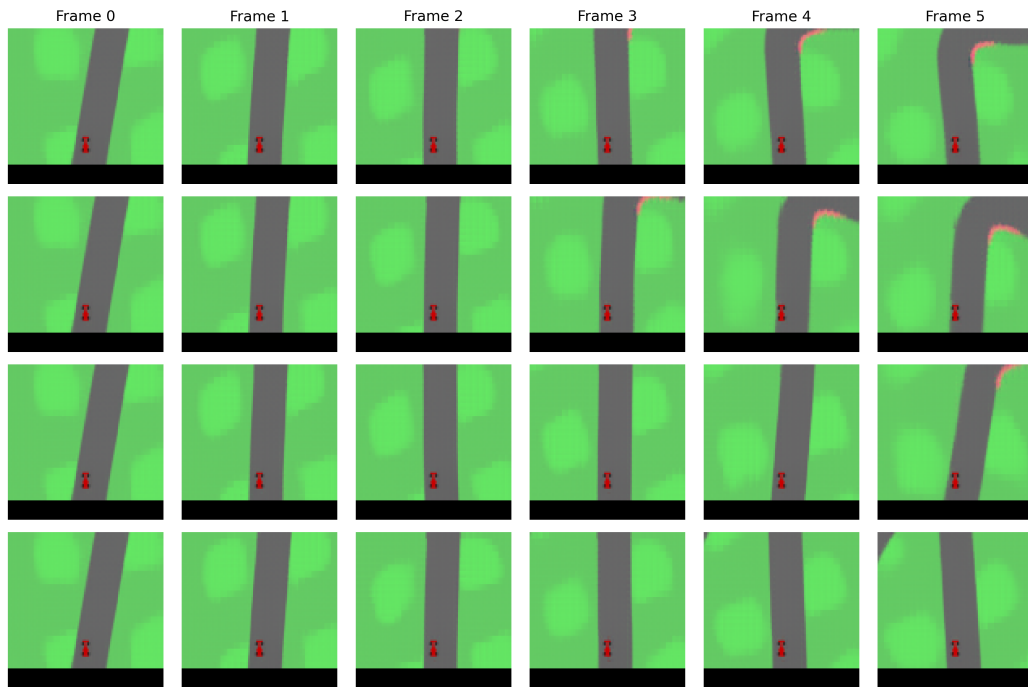


Figure 5: Diverse trajectories generated from the same starting state with 8 diffusion steps. Frame 0 is identical for all trajectories and serves as the initial state. Frames one through five are imagined by our dynamics model in latent space and then decoded. Our model shows the capacity to generate diverse futures.